CHAPTER 2

Test-Driven Development Basics

You have a problem.

You're the team leader for a development team that is distributed across multiple locations. You'd like to be able to maintain a common list of tasks for the team. For each task, you'd like to maintain data such as the status of the task, which pair of developers the task is assigned to, and so on. You'd also like to be able to use the past rate of task completion to estimate the project's completion date. For some reason, none of the existing tools that do this are suitable (work with me here, folks) and so you've decided to roll your own. You're calling it Gatherer.

As you sit down to start working on Gatherer, your impulse is going to be to start writing code immediately. That's a great impulse, and I'm just going to turn it about 10 degrees east. Instead of starting off by writing code, you're going to start off by writing tests.

In the introductory chapter, I talked about why you might work test-first. In this chapter you'll look at the basic mechanics of a TDD cycle. Your starting point will be to create some business logic with our models, because model logic is the easiest part of a Rails application to test—in fact, most of this chapter won't touch Rails at all. In the next chapter, you'll start testing the display parts of the Rails framework.

Infrastructure

First off, you'll need a Rails application. You'll be using Rails 5.2.0 and Ruby 2.5.0. Let's start by generating the Rails application from the command line:

```
% gem install rails
% mkdir gatherer
% cd gatherer
% rails new .
```